Reconstructing Global Models from Federated Client Weights Using Neural Networks

1st Arsh Javed
Computer Science and Engineering
Graphic Era Deemed to be University
Dehradun, India
arsh0javed@gmail.com

2nd Ashwini Kumar Singh
Computer Science and Engineering
Graphic Era Deemed to be University
Dehradun, India
ashwini.21mar@gmail.com

3rd Arun Chauhan
Computer Science and Engineering
Graphic Era Deemed to be University
Dehradun, India
arun.chauhan@geu.ac.in

***Abstract*—Federated Learning (FL) enables collaborative training across decentralized devices without sharing raw data. A persistent challenge is aggregating model updates from clients with heterogeneous, non-IID data distributions, where standard heuristic methods like FedAvg often underperform. We investigate the hypothesis that the aggregation function itself is *learnable*. Our framework trains separate Multi-Layer Perceptrons (MLPs) for each layer of a BERT-based model, learning to map client weight vectors to optimal global weights using a centralized oracle as the ground truth target. We partition parameters into 70% for training and 30% for reconstruction testing. The reconstructed model achieves 93.50% accuracy, substantially outperforming both FedAvg (83.98%) and FedMedian (83.89%), while approaching the centralized oracle (94.04%), with an average reconstruction fidelity of $R^2 = 0.953$. Although this proof-of-concept requires access to a ground truth target, it demonstrates that aggregation functions are learnable and that client parameters encode sufficient information for high-quality global model reconstruction.**

***Index Terms*—Federated Learning, Model Aggregation, Neural Networks, Parameter Reconstruction, Non-IID Data, BERT**

I. Introduction

Federated learning (FL) enables decentralized model training across multiple clients without centralized data collection, making it valuable in privacy-sensitive domains [1]. However, aggregating models trained on non-IID data remains a fundamental challenge. The variations in data distributions across clients create inconsistencies in model updates, which standard aggregation methods like FedAvg [1] and its variants [2], [13] often struggle to resolve [5].

Recent research has explored more sophisticated model fusion techniques to address these limitations. Approaches range from variance reduction methods like SCAFFOLD [15] and robust aggregation via coordinate-wise median [12], to learned aggregation weights (FedLAW) [7], matched averaging (FedMA) [14], model recombination (FedMR) [6], and layer-wise posterior aggregation (FedLPA) [9]. Recent work on personalized heterogeneous aggregation [16] further underscores the growing interest in moving beyond simple averaging. Building on prior work in neural network fusion via graph matching [4], these methods improve upon FedAvg, yet the fundamental question of whether the aggregation function itself is *learnable from data* remains underexplored. Recent advances in federated transfer learning for healthcare [17] and privacy-preserving FL for IoT [18] further highlight the breadth of FL applications and the need for improved aggregation strategies.

In this work, we explore precisely this question through a **reconstruction-based** experimental framework. We train a separate, specialized neural network for each parameter layer of a global model, learning to map client parameters to the global consensus. Rather than proposing an immediately deployable algorithm, we frame this work as a proof-of-concept investigation into the learnability of aggregation functions, using a centralized oracle as the training target.

Our key contributions are:

- We demonstrate that neural networks can reconstruct global model weights from client parameters with high fidelity ($R^2 = 0.953$), establishing that the aggregation function is learnable.
- We show that the reconstructed model achieves 93.50% accuracy, significantly outperforming heuristic baselines (FedAvg: 83.98%, FedMedian: 83.89%) and approaching the centralized oracle (94.04%).
- We provide a detailed parameter space analysis revealing that our learned reconstruction produces a more regularized model with lower L1/L2 norms, offering insights into how neural aggregation differs structurally from simple averaging.

II. Methodology

A. Problem Formulation

Consider K clients, each possessing a local model with parameters $\{W_1, \ldots, W_K\}$, where client k's model has been trained on its local data D_k. The aggregation task is to find a function $\mathcal{A}$ such that:

$$W_g = \mathcal{A}(W_1, W_2, \ldots, W_K) \tag{1}$$

where W_g represents the global model parameters. In FedAvg, $\mathcal{A}$ is a simple weighted mean. We hypothesize that there exists a *learned* function f_θ that better approximates the optimal global parameters. Decomposing the problem layer-wise, for each layer l, we learn:

$$\hat{w}_g^{(l)} = f_{\theta_l}(w_1^{(l)}, w_2^{(l)}, \ldots, w_K^{(l)}) \tag{2}$$

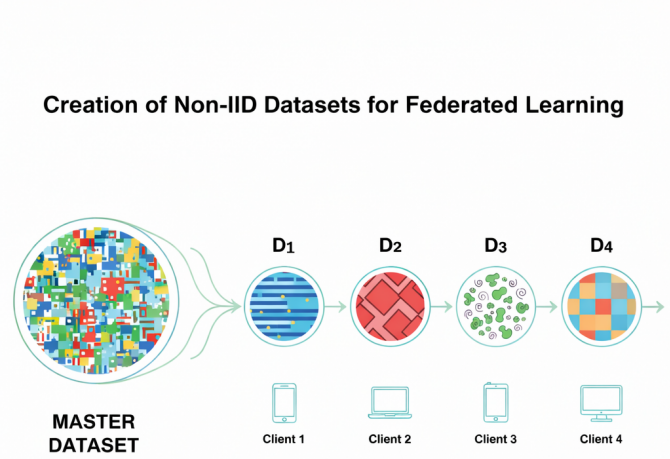


Fig. 1. Dataset preparation process showing the creation of four distinct datasets (D_1, D_2, D_3, D_4)

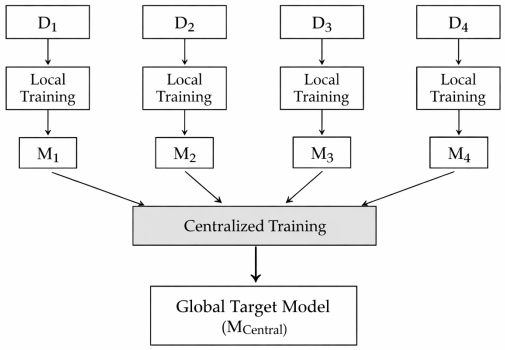


Fig. 2. Training process for client models (M_1–M_4) and centralized target model ($M_{Central}$).

where f_{θ_l} is a layer-specific MLP trained to minimize $\|\hat{w}_g^{(l)} - w_g^{(l)}\|^2$.

B. Ground Truth Generation

The first step is establishing a high-quality ground truth global model. Client models (M_1, M_2, M_3, M_4), each trained on distinct non-IID data, transmit their full weights to the server.

The server aggregates these weights to produce a global model. We employ Centralized Training, denoting the result $M_{Central}$, trained on $D_1 \cup D_2 \cup D_3 \cup D_4$. This model represents the ideal theoretical optimum and serves as the ground truth target for our learning-based aggregator.

C. Parameter Prediction for Model Reconstruction

We partition the global model's parameters into a training set (70%) and held-out evaluation set (30%). The neural networks are trained on the 70% partition to predict the remaining 30%, given corresponding client parameters as input.

For each layer, client parameters are fed as input to the corresponding prediction network. This setup validates the *reconstruction* capability rather than direct communication compression, serving as a foundation for future compression-oriented architectures.

D. A Layer-wise and Adaptive Framework

Our approach treats each parameter tensor of the global model as an independent prediction task. We train a unique prediction network for every qualifying layer in $M_{Central}$, capturing distinct relationships within different parts of the network.

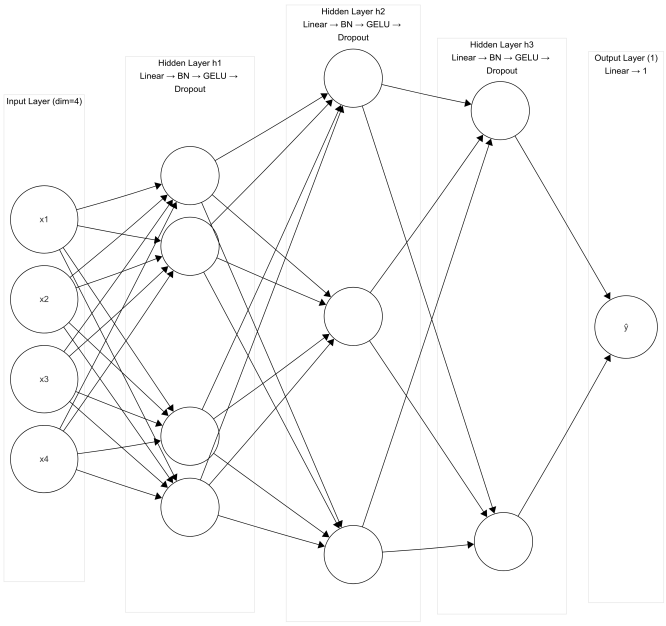


Fig. 3. Architecture of the neural network for weight mapping.

1) Adaptive Hyperparameter Tuning: Each prediction network is dynamically configured based on layer size. We calculate the 25th and 75th percentiles of parameter counts across all layers to establish thresholds for 'small', 'medium', and 'large' layers, assigning tailored hyperparameters (hidden dimensions, learning rate, dropout, weight decay) accordingly.

2) Network Architecture: Each prediction network is an MLP with input dimension equal to the number of clients (four). Each hidden block contains a Linear layer, `BatchNorm1d`, `GELU` activation, and Dropout. The final output layer is a single linear neuron predicting the target parameter value.

Algorithm 1 Layer-wise Weight Prediction Training

Require: Client models M_1, M_2, M_3, M_4; Target $M_{Central}$
Ensure: Trained prediction networks for each layer
for each layer l in $M_{Central}$ **do**
  $X \leftarrow$ Stack params from $M_1, ..., M_4$ at layer l
  $y \leftarrow$ Params from $M_{Central}$ at layer l
  $X_{tr}, X_{te}, y_{tr}, y_{te} \leftarrow$ Split(X, y, 0.7)
  Scale X_{tr}, y_{tr} using RobustScaler
  Init MLP with adaptive hyperparameters
  for epoch $= 1$ to 20 **do**
    $\hat{y} =$ MLP(X_{tr}); $\mathcal{L} =$ MSE($\hat{y}, y_{tr}$)
    Update via AdamW with gradient clipping
  end for
  Reconstruct layer l: merge y_{tr} with MLP(X_{te})
end for

E. Training the Prediction Networks

For each layer, training data is constructed at the individual parameter level. For each weight or bias term, we create a feature vector containing that parameter's value from each client model. The corresponding $M_{Central}$ parameter serves as the ground truth. The dataset is partitioned with 70% for training (X_{train}, y_{train}) and 30% for inference (X_{test}).

Data Scaling: To handle the varying scales of weights and improve training stability, we apply a `RobustScaler` to both the input features and the target values. This scaler is resilient to outliers, which are common in model weight distributions.

Loss Function and Optimization: We train each network by minimizing the Mean Squared Error (MSE) loss between its predictions and the true parameter values. We employ the `AdamW` optimizer and utilize a `CosineAnnealingLR` scheduler over 20 epochs to smoothly adjust the learning rate, promoting convergence to a more stable minimum.

Training Efficiency: To accelerate the process, training is conducted with mixed-precision (AMP) on CUDA-enabled devices. We also apply gradient clipping with a max norm of 1.0 to prevent exploding gradients and further stabilize training.

The complete training procedure is formalized in Algorithm 1.

Merging Strategy: The final model M_{Final} is reconstructed layer-by-layer by combining the 70% training weights from $M_{Central}$ with the 30% predicted by our network.

III. Experimental Setup and Results

A. Experimental Setup

Models and Datasets: The experiments involved four client models based on the *mental-bert-base-uncased* architecture [11], [22], a BERT model [10], [21] pre-trained on mental health text. The downstream task is binary depression classification from social media text. Each client model was fine-tuned on a distinct, non-overlapping subset of the data (D_1, D_2, D_3, D_4), simulating a realistic federated scenario where different institutions hold separate patient cohorts. To create a global target model, we trained a Centralized Model ($M_{Central}$) on the union of all client datasets, which serves as the benchmark for our parameter prediction method. This setup is motivated by recent advances in applying deep learning to mental health detection [19] and federated transfer learning for healthcare [17].

Client Model Training: Each client was trained for 5 epochs with AdamW (lr=2×10^{-5}, weight decay=0.01), linear scheduler (10% warmup), batch size 32, gradient accumulation over 2 steps, FP16 mixed-precision, balanced class weights in CrossEntropyLoss, and early stopping (patience=3) on validation loss.

Data Preprocessing: Input text for all models was preprocessed by lowercasing, removing URLs, mentions, and hashtags, and then tokenizing to a maximum sequence length of 128. Datasets were split into 70% for training, 15% for validation, and 15% for testing [8].

Aggregation Baselines: We compare against two aggregation baselines: (1) **FedAvg** [1], the standard element-wise weighted mean aggregation, and (2) **FedMedian** [12], which computes the coordinate-wise median across client parameters. Both baselines are computed as post-hoc aggregation of the same client weights used by our method, ensuring a fair comparison.

Reproducibility: To support reproducibility, our implementation code, trained models, and evaluation scripts will be released upon acceptance.

B. Prediction Network Configuration

Each network was configured dynamically based on layer size, trained for **20 epochs**:

- **Small Layers** (fewer than 412,876 parameters): Learning rate of 5×10^{-5}, hidden layers of [64, 32], dropout of 0.5, and weight decay of 1×10^{-4}.
- **Medium Layers**: Learning rate of 1×10^{-4}, hidden layers of [128, 64, 32], dropout of 0.3, and weight decay of 1×10^{-5}.
- **Large Layers** (more than 1,651,507 parameters): Learning rate of 2×10^{-4}, hidden layers of [256, 128, 64], dropout of 0.15, and weight decay of 1×10^{-6}.

A `CosineAnnealingLR` scheduler managed the learning rate over the training period.

C. Metrics

- Accuracy: Measures the classification performance of the final merged model against ground truth labels.
- Mean Squared Error (MSE): Tracks the prediction loss during training, defined as:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{3}$$

  where n is the number of parameters, y_i represents actual target weights, and $\hat{y}_i$ represents predicted weights.

- R^2 Score (Coefficient of Determination): Evaluates the goodness of fit for the weight prediction, calculated as:

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2} \quad (4)$$

where y_i are the actual target weights, $\hat{y}_i$ are the predicted weights, and $\bar{y}$ is the mean of the actual weights. An R^2 score closer to 1 indicates a better fit.

Each client model was evaluated on its local test dataset (Table I).

TABLE I
PERFORMANCE OF INDIVIDUAL CLIENT MODELS

Client Model	Test Accuracy	Test F1-Score
M_1	0.9810	0.9810
M_2	0.9822	0.9823
M_3	0.8658	0.8963
M_4	0.9064	0.9066

Test accuracies range from 86.58% to 98.22%, confirming the non-IID nature of client data and the challenge for standard aggregation.

TABLE II
PERFORMANCE COMPARISON OF AGGREGATION METHODS

Metric	FedAvg (M_{FedAvg})	FedMedian (M_{Median})	Central ($M_{Central}$)	Reconstructed (M_{Final})
Accuracy	83.98%	83.89%	94.04%	93.50%
F1-Score	83.95%	83.84%	94.04%	93.50%
R^2 Score	–	–	–	0.953

D. Performance Analysis

The reconstructed model achieved a classification accuracy of 93.50%, as detailed in Table II. This represents a substantial improvement over both heuristic baselines—FedAvg (83.98%) and FedMedian (83.89%)—and achieves 99.4% of the centralized oracle's accuracy (93.50% / 94.04%). Notably, FedMedian performs comparably to FedAvg (83.89% vs. 83.98%), which is expected in a non-adversarial setting with only four honest clients. This confirms that the neural aggregator successfully learned a mapping that approximates the true global optimum, correcting for the information loss typically associated with simple heuristic aggregation [12].

The high average R^2 score of 0.953 indicates strong correlation between predicted and actual weights, demonstrating that client parameters encode sufficient information to reconstruct the centralized optimum, validating the "learnability" hypothesis.

Per-layer analysis shows most transformer block weights achieved $R^2 > 0.99$, though small layers (e.g., `token_type_embeddings`, `classifier.weight`) with only 1,075 samples yielded negative R^2 scores. Despite these outliers, exceptional performance on the majority of parameters led to successful reconstruction.

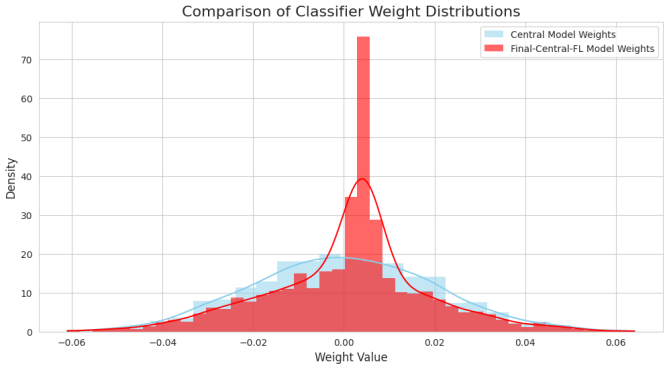


Fig. 4. Classifier layer weight distributions: M_{Final} shows a sharper, more regularized peak than $M_{Central}$.

E. Generalizability Verification

To verify generalizability, we trained the prediction networks to reconstruct M_{FedAvg} instead of $M_{Central}$. The results achieved $R^2 = 0.965$, even higher than the Central reconstruction (0.953), as expected since FedAvg is a simpler averaging function. This confirms the framework can learn different aggregation rules, not just the centralized optimum.

F. Parameter Space Analysis

We analyzed structural differences between M_{FedAvg} and M_{Final} beyond performance metrics, examining weight structure, parameter norms, and centrality to clients.

1. Weight Structure and Sparsity: The classifier layer weight distributions (Fig. 4) show M_{Final} has a more pronounced peak at zero (leptokurtic) compared to $M_{Central}$'s smoother distribution. M_{Final} exhibits higher sparsity (2.14% vs. 1.95% of weights below 10^{-3}), suggesting our framework prunes less impactful parameters.

2. Model Regularization via Norms: As shown in Table III, M_{Final} achieves lower L1 and L2 norms than $M_{Central}$. Lower L2 norms are associated with better generalization, providing evidence that our aggregator acts as intelligent regularization.

TABLE III
L1 AND L2 NORM COMPARISON OF GLOBAL MODELS

Model	L1 Norm	L2 Norm
$M_{Central}$	3.739×10^6	471.84
M_{Final}	3.708×10^6	468.04

3. Centrality to Client Models: Cosine similarity analysis (Fig. 5) shows both $M_{Central}$ (avg: 0.9976) and M_{Final} (avg: 0.9975) are highly similar to all clients. The minimal drop confirms that the **predicted 30% of weights aligned well** with the clients' geometric center.

G. Practical Implications

The demonstrated learnability has concrete implications: (1) **Meta-Aggregation**—given enough historical FL rounds, an aggregation network could be pre-trained and fine-tuned for

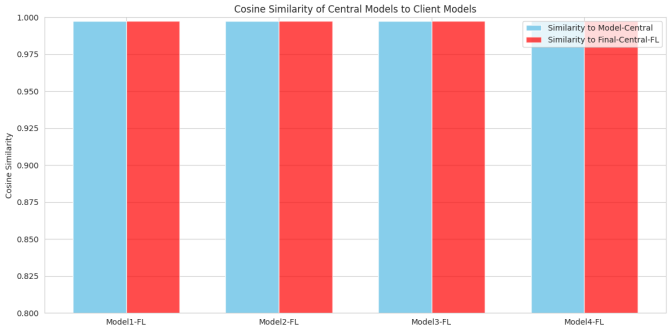


Fig. 5. Cosine similarity of global models to each client. Both are highly central.

new tasks, performing meta-learning over aggregation itself; (2) **Intrinsic Quality Metrics**—the R^2 score serves as a per-layer diagnostic, enabling adaptive hybrid aggregation where low-R^2 layers are flagged for alternative treatment.

H. Limitations

The primary limitation is the need for a pre-existing ground truth model ($M_{Central}$), requiring centralized data access or a prior communication round. Additionally, the architecture requires full client weights (no communication reduction), evaluates in a one-shot setting (convergence over multiple rounds is unexplored), and assumes homogeneous client architectures [3].

Small layers with few parameters (e.g., 1,075 samples) yielded negative R^2 scores, and training separate networks per layer is computationally expensive. Despite these limitations, our results establish that aggregation functions are learnable, providing a foundation for practical learned aggregation.

IV. Conclusion and Future Work

A. Conclusion

We introduced a reconstruction-based framework to investigate the learnability of the aggregation function in Federated Learning. By replacing heuristic averaging with layer-specific neural networks, we demonstrated that client parameters encode sufficient information to reconstruct the global model with high fidelity ($R^2 = 0.953$). The reconstructed model achieves 93.50% accuracy, significantly outperforming both FedAvg (83.98%) and FedMedian (83.89%), and approaching the centralized oracle (94.04%). This result establishes that non-linear, learned aggregation can capture relationships in the parameter space that both mean- and median-based heuristics miss, opening a promising direction for intelligent model fusion and reconstruction.

B. Future Work

Key directions include: (1) *Cross-architecture generalization* via meta-learning for architecture-agnostic weight transformation; (2) *Communication-efficient architectures* that predict parameters from partial inputs; (3) *Privacy enhancements* through differential privacy or secure computation, building on recent work in privacy-preserving FL [18] and federated vs. centralized comparisons [20]; and (4) *Improved small-layer prediction* using simpler regression models or adaptive exclusion strategies.

References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Aguera y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *AISTATS*, 2017.

[2] T. Li, A. K. Sahu, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *MLSys*, 2020.

[3] Q. Li, Z. Wen, and B. He, "Practical federated gradient boosting decision trees," in *AAAI*, 2020.

[4] C. Liu, C. Lou, R. Wang, A. Y. Xi, L. Shen, and J. Yan, "Deep neural network fusion via graph matching with applications to model ensemble and federated learning," in *ICML*, 2022.

[5] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," arXiv preprint arXiv:1806.00582, 2018.

[6] M. Hu, Z. Yue, X. Xie, C. Chen, Y. Huang, X. Wei, X. Lian, Y. Liu, and M. Chen, "FedMR: Federated learning via model recombination," in *KDD*, 2024.

[7] Z. Li, T. Lin, X. Shang, and C. Wu, "Revisiting weighted aggregation in federated learning with neural networks," in *ICML*, 2023.

[8] Divya Agarwal, Vijay Singh, Ashwini Kumar Singh, and Parul Madan, "Dwarf Updated Pelican Optimization Algorithm for Depression and Suicide Detection from Social Media," in *Psychiatric Quarterly*, Volume 96, pages 529–562, (2025).

[9] X. Liu, et al., "FedLPA: Layer-Wise Posterior Aggregation for One-Shot Federated Learning," in *NeurIPS*, 2024.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *NAACL-HLT*, 2019.

[11] S. Ji, T. Zhang, L. Ansari, J. Fu, P. Tiwari, and E. Cambria, "MentalBERT: Publicly available pretrained language models for mental healthcare," in *LREC*, 2022.

[12] D. Yin, Y. Chen, R. Kannan, and P. Bartlett, "Byzantine-robust distributed learning: Towards optimal statistical rates," in *ICML*, 2018.

[13] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *NeurIPS*, 2020.

[14] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *ICLR*, 2020.

[15] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *ICML*, 2020.

[16] J. Wang, Q. Li, L. Lyu, and F. Ma, "pFedClub: Controllable heterogeneous model aggregation for personalized federated learning," in *NeurIPS*, 2024.

[17] V. Khullar, P. Kaur, S. Gargrish, A. M. Mishra, P. Singh, M. Diwakar, A. Bijalwan, and I. Gupta, "Minimal sourced and lightweight federated transfer learning models for skin cancer detection," *Scientific Reports*, vol. 15, 2025.

[18] A. Suryavanshi, S. Mehta, M. Kumar, and S. Thapliyal, "Transforming IoT with Federated Multimodal Learning: A Privacy-Centric Approach," in *2024 4th Int. Conf. Intelligent Technologies (CONIT)*, IEEE, 2024.

[19] S. M. Padmaja, S. R. Godla, J. V. N. Ramesh, E. Muniyandy, P. Sridevi, Y. A. B. El-Ebiary, and D. N. P. Devadhas, "Depression detection in social media using NLP and hybrid deep learning models," *Int. J. Adv. Comput. Sci. Appl. (IJACSA)*, vol. 16, no. 2, 2025.

[20] N. Subramanian, L. Ravi, M. J. Shaan, M. Devarajan, T. Choudhury, K. Kotecha, and S. Vairavasundaram, "Securing mobile devices from malware: A faceoff between federated learning and deep learning models for Android malware classification," *J. Comput. Sci.*, vol. 20, no. 2, pp. 254–264, 2024.

[21] Ashwini Kumar Singh, Vahid Ghafouri, Jose Such, and Guillermo Suarez-Tangil, "Differences in the toxic language of cross-platform communities," in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 18, pp. 1463-1476. 2024.

[22] Kanishka Bhatt, Ashwini Kumar Singh, Priyank Pandey, Ankit Vishnoi, and Preeti Narooka, "Cross platform social media analysis for mental health detection, in "*Discover Mental Health*, 2026.